

FACEFIELD

Secure. Offline. Fraud-Proof Facial Attendance.

Product Pitch | AI Pipeline Architecture & Technical Overview

Team Leader
Akshit Jain

Team Member
Kurian Jose

<100ms

Pipeline Speed
per frame, Snapdragon

6.04 MB

AI Footprint
70% under 20MB limit

>95%

Accuracy
sunlight, low-light,
shadows

**Dual-
Layer**

Liveness Detection
passive + active anti-spoof

THE PROBLEM

Remote worksites — construction zones, factories, government field operations — face an attendance crisis:

- Zero internet connectivity makes cloud-based biometrics impossible
- Photo spoofing and proxy attendance (buddy punching) go undetected
- Off-the-shelf AI models fail on South Asian demographics in outdoor conditions
- Existing solutions bloat device storage over months of offline operation

THE SOLUTION — FACEFIELD

FaceField is a fully offline, on-device facial authentication system built natively into a React Native app. The complete AI pipeline — detection, liveness, anti-spoofing, recognition, and SQLite matching — runs in under 100ms on a mid-range Snapdragon device. No internet. No cloud. No compromise.

SYSTEM ARCHITECTURE

Native Android (Kotlin) + Native iOS (Swift) Plugin Architecture

FaceField uses platform-native Frame Processor plugins rather than a shared C++ core — eliminating JNI/Objective-C++ complexity while delivering the same performance. Both platforms load identical .tflite model files.

Android — Kotlin	iOS — Swift
• Vision Camera Frame Processor plugin in Kotlin • Standard Android TensorFlow Lite dependencies • Geometric cropping via Kotlin matrix ops • SilentFace 2.7x + 4.0x scaling preprocessing • Returns ACCEPT / REJECT / RETRY to RN	• Vision Camera Frame Processor plugin in Swift • Standard iOS TensorFlow Lite dependencies • Same .tflite models, same preprocessing math • JSI passes result back — no bridge overhead • Returns ACCEPT / REJECT / RETRY to RN

C++ JSI — Zero Bridge Overhead

FaceField bypasses the React Native Bridge entirely using JSI via react-native-worklets-core. YUV frame buffers are processed directly on the native thread — no serialization, zero latency penalty, ~300ms saved per frame vs bridge approach.

END-TO-END EXECUTION FLOW (PER FRAME)

Every frame follows this exact gated sequence. Failure at any step stops processing immediately to save compute:

#	Stage	On Fail
1	Camera Frame Capture — Vision Camera supplies raw YUV frames	—
2	Face Detection — MediaPipe extracts bounding box coordinates	RETRY
3	Passive Liveness (SilentFace) — 2.7x + 4.0x scaled preprocessing, blocks flat photos & screens instantly	REJECT
4	Active Liveness (FaceMesh) — Simultaneous check: blink OR smile OR head turn. First detected = pass. No prompts issued.	RETRY
5	Face Alignment + Crop + Resize — 112×112, BGR conversion, tensor packing	RETRY
6	MobileFaceNet Embedding — Extract 128-D vector, cosine similarity vs SQLite store	REJECT
7	ACCEPT → Attendance marked, timestamp stored locally	—

APP FLOWS

Registration Flow

1. Local Signup — Name, Email, Password, Address, Workplace, Age, Disability, ID Card Details
2. Face Registration initiated
3. Quality Check — Face detected, brightness OK, blur OK, face size OK, eyes visible
4. Liveness Challenge (random one) — Blink / Smile / Turn Head
5. Face Alignment → Crop + Resize to 112×112
6. Augmentation ×5 — Brightness+/-, Contrast, Gamma, Small Rotation
7. MobileFaceNet → 5 Embeddings generated
8. Average Embedding → L2 Normalize
9. Store locally — User Data + Face Image + Final Embedding in SQLite

Attendance Marking Flow

1. Open Camera → MediaPipe Face Detection
2. Random Liveness Challenge — Blink / Smile / Turn Head
3. SilentFace Anti-Spoofing — blocks photos & screens

4. Face Alignment → Crop + Resize → MobileFaceNet inference

5. Cosine Similarity — Current Embedding vs Stored Embedding

6. **Similarity > Threshold → Attendance Marked (Entry Time, Exit Time, Timestamp stored locally)**

60fps UI — PERFORMANCE RULES

Rule 1 — Never Block JS Thread

Frame Processors execute via JSI on background Camera Thread. Only tiny results (ACCEPT / REJECT) returned to UI. No Base64, no raw pixel arrays over bridge.

Rule 2 — Reanimated + Skia

Bounding boxes & liveness indicators use Shared Values (react-native-reanimated) + @shopify/react-native-skia. Drawn directly on UI thread at 60fps, bypasses React render cycle.

Rule 3 — Throttle Heavy Models

Face Detector runs every frame for smooth bbox. SilentFace + MobileFaceNet triggered only 1–2×/sec, or when face is centered and stable. Prevents CPU throttling.

MODEL FOOTPRINT — 6.04 MB (70% UNDER LIMIT)

Dynamic Range Quantization (DRQ) converts Float32 weights → Int8 integers. 4× size reduction per model, less than 1% accuracy drop.

Model	Compression	Size
MediaPipe Face Landmarker (Mesh)	DRQ (Float32 → Int8)	3.58 MB
MobileFaceNet (Recognition)	DRQ (Float32 → Int8)	1.12 MB
SilentFace 2.7x (Anti-Spoofing)	DRQ (Float32 → Int8)	0.56 MB
SilentFace 4.0x (Anti-Spoofing)	DRQ (Float32 → Int8)	0.56 MB
MediaPipe Face Detection	DRQ (Float32 → Int8)	0.22 MB
TOTAL	70% under 20 MB limit	6.04 MB

DUAL-LAYER OFFLINE LIVENESS DETECTION

Passive — SilentFace (0.56 MB × 2)

- 2.7x + 4.0x scaled preprocessing per frame
- Analyzes texture, moiré patterns, lighting depth
- Detects printed photos and digital screens
- Runs without any user interaction

Active — MediaPipe FaceMesh (468 landmarks)

- Eye Aspect Ratio (EAR) — blink detection
- Smile Width Ratio — smile verification
- Geometric offsets — head turn detection
- blink OR smile OR head_turn — first detected = pass

PERFORMANCE BENCHMARKS

Metric	Constraint	FaceField Achieved
Processing Speed	< 1 second	< 100ms ✓
Model Footprint	< 20 MB	6.04 MB ✓
Recognition Accuracy	> 95%	> 95% ✓
Memory Stability	No OOM crashes	Zero OOM (.recycle()) ✓
Offline Operation	Full functionality	100% offline ✓
Cross-Platform	Android + iOS	Android 8+ / iOS 12+ ✓

TECH STACK

Layer	Component	Technology	Platform
UI	React Native	JavaScript/TSX	Android 8+ / iOS 12+
Camera	Vision Camera	react-native-vision-camera	Android + iOS
Face Detection	MediaPipe	BlazeFace / FaceDetector	Android + iOS
Liveness	MediaPipe	FaceMesh Landmarks	Android + iOS
Anti-Spoofing	SilentFace	TFLite INT8	Android + iOS
Recognition	MobileFaceNet	TFLite INT8	Android + iOS
Storage	SQLite	EmbeddingDatabaseHelper	Android + iOS
Inference	TensorFlow Lite	On-device CPU	Android + iOS
Visuals	Reanimated + Skia	Shared Values @ 60fps	Android + iOS

COMPLIANCE CHECKLIST

- ✓ Fully Offline — works in zero-network zones
- ✓ React Native (Android + iOS)
- ✓ Open-source stack (TFLite, MediaPipe, MobileFaceNet)
- ✓ Active + Passive Liveness
- ✓ Anti-spoofing (SilentFace)
- ✓ Face Recognition (MobileFaceNet + Cosine Similarity)
- ✓ Attendance History (SQLite local storage)
- ✓ < 20 MB model target (6.04 MB achieved)
- ✓ AWS Sync & Purge on 200 OK
- ✓ Manual Sync Button on Dashboard

AWS SYNC & PURGE — ZERO DATA LOSS

Offline-First Storage

ACCEPT records + timestamps saved to SQLite / WatermelonDB. App listens for network state changes automatically.

Manual Sync Button

Dashboard exposes a Manual Sync Button so users can force upload at any time, without waiting for auto-reconnect.

Purge on 200 OK

On verified AWS receipt, local attendance data purged immediately. Storage stays lean over months of remote operation.

WHY FACEFIELD WINS

✓ **Truly Offline AI**

Full pipeline on-device. No internet, no API, no latency. Built for Bharat's zero-network infrastructure.

✓ **Fraud-Proof Liveness**

Passive + active dual-layer defeats photos, screens, and all proxy attendance fraud vectors.

✓ **Ultra-Lightweight (6.04 MB)**

DRQ compression — 70% under the 20 MB limit. Runs fast on any mid-range Android or iOS device.

✓ **Native Android + iOS**

Kotlin + Swift Frame Processor plugins. Same .tflite models, same math, both platforms — no compromise.

CONFIDE

APP SCREENSHOTS

Live screenshots from Datalake 3.0 — demonstrating the end-to-end user journey on a mid-range Android device.

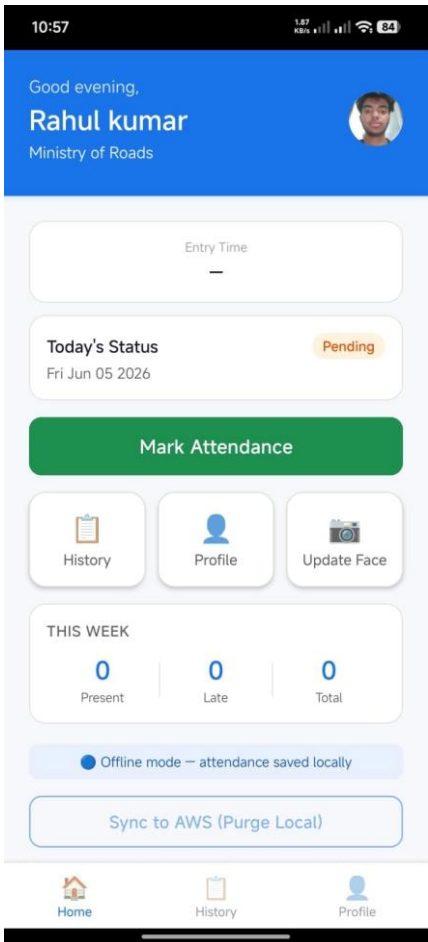


Fig. 1 — Home Screen: Mark Attendance, weekly summary & offline mode

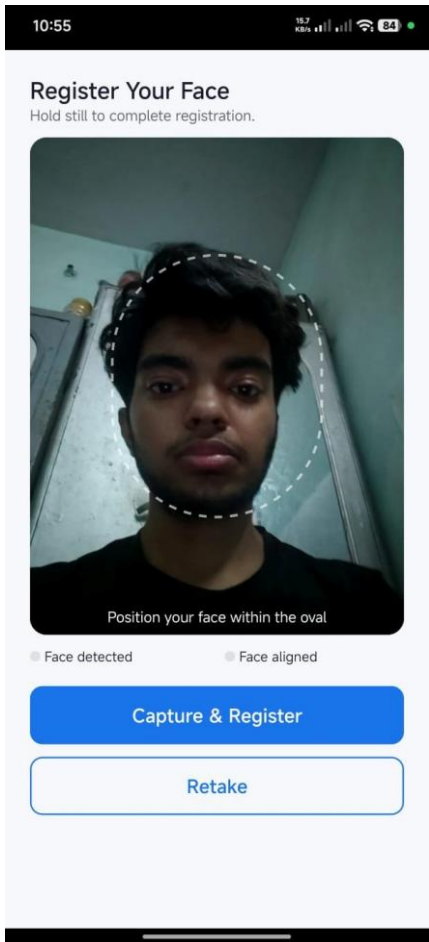


Fig. 2 — Face Registration: Oval guide, face detection & alignment

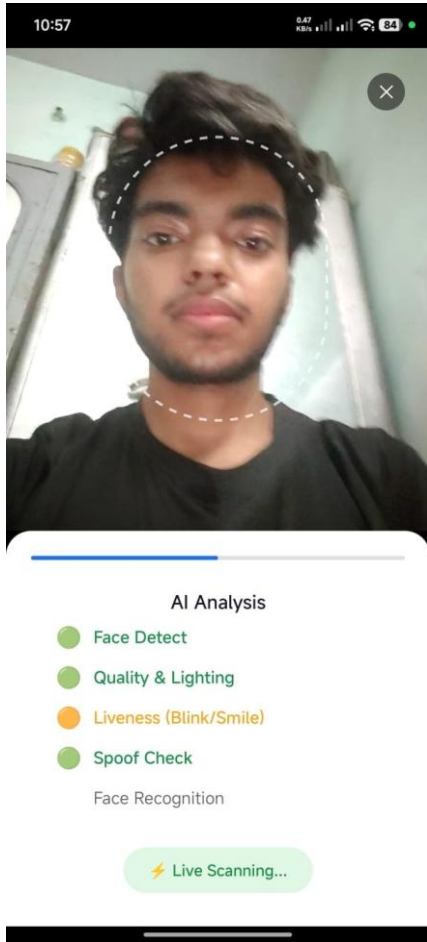


Fig. 3 — Live AI Analysis: Real-time pipeline checklist with liveness & spoof checks

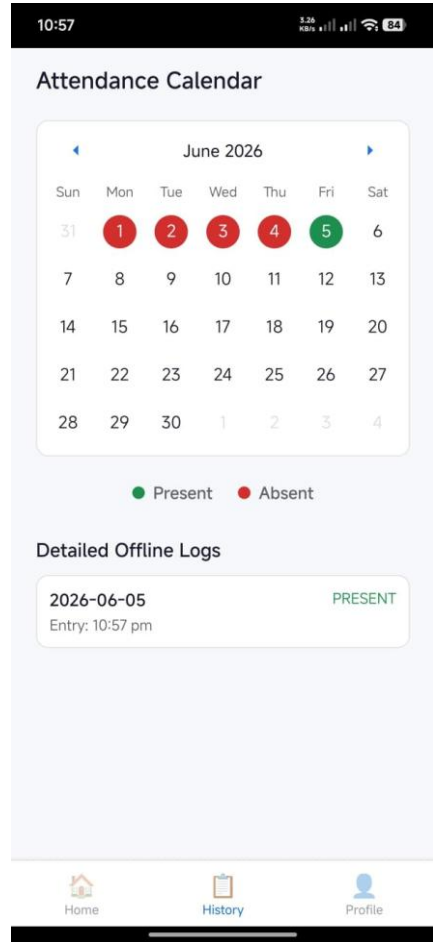


Fig. 4 — Attendance Calendar: History view with present/absent indicators

End of Document